

TOPIC 4 : DYNAMIC PROGRAMMING

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

```
[ ] def dice_ways(num_dice, num_sides, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]

    dp[0][0] = 1

    for d in range(1, num_dice + 1):
        for s in range(1, target + 1):
            for face in range(1, num_sides + 1):
                if s >= face:
                    dp[d][s] += dp[d - 1][s - face]

    return dp[num_dice][target]

# Test Case 1
print("Test Case 1:")
print("Number of ways to reach sum 7:",
      dice_ways(2, 6, 7))

# Test Case 2
print("Test Case 2:")
print("Number of ways to reach sum 10:",
      dice_ways(3, 4, 10))

v Test Case 1:
   Number of ways to reach sum 7: 6
   Test Case 2:
   Number of ways to reach sum 10: 6
```

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

```
Q Commands + Code + Text ▶ Run all

[ ] [ ]
dp1[i] = min(
    dp1[i - 1] + a1[i],
    dp2[i - 1] + t2[i - 1] + a1[i]
)

dp2[i] = min(
    dp2[i - 1] + a2[i],
    dp1[i - 1] + t1[i - 1] + a2[i]
)

return min(dp1[n - 1] + x1, dp2[n - 1] + x2)

# Example
a1 = [4, 5, 3, 2]
a2 = [2, 10, 1, 4]

t1 = [7, 4, 5]
t2 = [9, 2, 8]

e1 = 10
e2 = 12

x1 = 18
x2 = 7

result = assembly_line(a1, a2, t1, t2, e1, e2, x1, x2)

print("Minimum time required:", result)

... Minimum time required: 35
```

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.

```
[ ]
    for i in range(lines):
        dp[i][0] = times[i][0]

    # Remaining stations
    for j in range(1, stations):
        for i in range(lines):
            dp[i][j] = min(
                dp[k][j - 1] + transfer[k][i]
                for k in range(lines)
            ) + times[i][j]

    return min(dp[i][stations - 1] for i in range(lines))

# Input
times = [
    [5, 9, 3],
    [6, 8, 4],
    [7, 6, 5]
]

transfer = [
    [0, 2, 3],
    [2, 0, 4],
    [3, 4, 0]
]

result = three_assembly_lines(times, transfer)

print("Minimum production time:", result)
```

... Minimum production time: 17

4. Write a c program to find the minimum path distance by using matrix form.

```
[ ]
    from itertools import permutations

    def tsp(matrix):
        n = len(matrix)
        start = 0
        minimum = float('inf')

        for path in permutations(range(1, n)):
            distance = matrix[start][path[0]]

            for i in range(len(path) - 1):
                distance += matrix[path[i]][path[i + 1]]

            distance += matrix[path[-1]][start]
            minimum = min(minimum, distance)

        return minimum

    matrix = [
        [0, 10, 15, 20],
        [10, 0, 35, 25],
        [15, 35, 0, 30],
        [20, 25, 30, 0]
    ]

    print("Minimum path distance:", tsp(matrix))
```

... Minimum path distance: 80

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.

```
from itertools import permutations

def tsp(matrix):
    n = len(matrix)
    start = 0
    minimum = float('inf')

    for path in permutations(range(1, n)):
        distance = matrix[start][path[0]]

        for i in range(len(path) - 1):
            distance += matrix[path[i]][path[i + 1]]

        distance += matrix[path[-1]][start]
        minimum = min(minimum, distance)

    return minimum

matrix = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum path distance:", tsp(matrix))
```

```
... Minimum path distance: 80
```

6. Given a string *s*, return the longest palindromic substring in *S*.

```

def longest_palindrome(s):
    if len(s) < 2:
        return s

    start = 0
    end = 0

    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
            left -= 1
            right += 1
        return left + 1, right - 1

    for i in range(len(s)):
        l1, r1 = expand(i, i)
        l2, r2 = expand(i, i + 1)

        if r1 - l1 > end - start:
            start, end = l1, r1

        if r2 - l2 > end - start:
            start, end = l2, r2

    return s[start:end + 1]

print(longest_palindrome("babad"))
print(longest_palindrome("cbbd"))

```

```

... bab
bb

```

7. Given a string *s*, find the length of the longest substring without repeating characters.

```
def longest_substring(s):
    seen = {}
    left = 0
    maximum = 0

    for right in range(len(s)):
        if s[right] in seen and seen[s[right]] >= left:
            left = seen[s[right]] + 1

        seen[s[right]] = right
        maximum = max(maximum, right - left + 1)

    return maximum

print(longest_substring("abcabcbb"))
print(longest_substring("bbbbbb"))
print(longest_substring("pwwkew"))
```

```
3
1
3
```

8. Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words. Note that the same word in the dictionary may be reused multiple times in the segmentation.

```
def word_break(s, wordDict):
    words = set(wordDict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break

    return dp[len(s)]

print(word_break("leetcode", ["leet", "code"]))
print(word_break("applepenapple", ["apple", "pen"]))
print(word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
```

```
... True
True
False
```

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango }

```

def can_segment(s, dictionary):
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for word in dictionary:
            if i >= len(word) and dp[i - len(word)]:
                if s[i - len(word):i] == word:
                    dp[i] = True
                    break

    return dp[-1]

dictionary = [
    "i", "like", "sam", "sung", "samsung",
    "mobile", "ice", "cream", "icecream",
    "man", "go", "mango"
]

print("ilike:", "Yes" if can_segment("ilike", dictionary) else "No")
print("ilikesamsung:", "Yes" if can_segment("ilikesamsung", dictionary) else "No")

```

```

... ilike: Yes
    ilikesamsung: Yes

```

10. Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' '; when necessary so that each line has exactly maxWidth characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left-justified, and no extra space is inserted between words. A word's length is guaranteed to be greater than 0 and not exceed maxWidth. The input array words contains at least one word.



```
line_words = words[i:j]
spaces = maxWidth - length
gaps = len(line_words) - 1

if j == len(words) or gaps == 0:
    line = " ".join(line_words)
    line += " " * (maxWidth - len(line))
else:
    each = spaces // gaps
    extra = spaces % gaps

    line = ""
    for k in range(gaps):
        line += line_words[k]
        line += " " * (each + (1 if k < extra else 0))

    line += line_words[-1]

result.append(line)
i = j

return result

words = ["This", "is", "an", "example", "of", "text", "justification."]
for line in full_justify(words, 16):
    print(line)
```

```
... This    is    an
   example of text
 justification.
```

11. Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the WordFilter class: WordFilter(string[] words) Initializes the object with the words in the dictionary. f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

```

class WordFilter:

    def __init__(self, words):
        self.words = words

    def f(self, pref, suff):
        for i in range(len(self.words) - 1, -1, -1):
            word = self.words[i]

            if word.startswith(pref) and word.endswith(suff):
                return i

        return -1

words = ["apple"]
obj = WordFilter(words)

print(obj.f("a", "e"))

```

... 0

12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path
 Input: $n = 4$, $edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]$, distance Threshold = 4.

```

▶ edges = [
    [0, 1, 3],
    [1, 2, 1],
    [1, 3, 4],
    [2, 3, 1]
]

dist = floyd_warshall(4, edges)

count = []
threshold = 4

for i in range(4):
    count.append(sum(
        1 for j in range(4)
        if i != j and dist[i][j] <= threshold
    ))

print("City:", max(range(4), key=lambda x: (-count[x], x)))

```

```

... Before Floyd:
[0, 3, inf, inf]
[3, 0, 1, 4]
[inf, 1, 0, 1]
[inf, 4, 1, 0]

```

```

After Floyd:
[0, 3, 4, 5]
[3, 0, 1, 2]
[4, 1, 0, 1]
[5, 2, 1, 0]
City: 3

```

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.

```

▶ for i in range(6):
    matrix[i][i] = 0

edges = [
    (0, 1, 1),
    (0, 2, 5),
    (1, 2, 2),
    (1, 3, 1),
    (2, 4, 3),
    (3, 4, 1),
    (3, 5, 6),
    (4, 5, 2)
]

for u, v, w in edges:
    matrix[u][v] = w
    matrix[v][u] = w

before = floyd(matrix)

print("A to F before failure:", before[0][5])

# B-D link failure
matrix[1][3] = INF
matrix[3][1] = INF

after = floyd(matrix)

print("A to F after failure:", after[0][5])

```

```

... A to F before failure: 5
    A to F after failure: 8

```

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm.

```

1      dist[i][j],
      dist[i][k] + dist[k][j]
    )

    answer = -1
    minimum = n

    for i in range(n):
        count = sum(
            1 for j in range(n)
            if i != j and dist[i][j] <= threshold
        )

        if count <= minimum:
            minimum = count
            answer = i

    return answer

edges = [
    [0, 1, 2],
    [0, 4, 8],
    [1, 2, 3],
    [1, 4, 2],
    [2, 3, 1],
    [3, 4, 1]
]

print(find_city(5, edges, 2))

```

✓ ... 0

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and frequencies. Execute your code and display the resulting OBST and its cost. Print the cost and root matrix.

```
Commands + Code + Text ▶ Run all ☁

[ ]
    root[i][j] = r

    return cost, root

keys = ['A', 'B', 'C', 'D']
freq = [0.1, 0.2, 0.4, 0.3]

cost, root = obst(keys, freq)

print("Cost:", cost[0][-1])
print("Cost Matrix:")

for row in cost:
    print(row)

print("Root Matrix:")

for row in root:
    print(row)

Cost: 1.7
Cost Matrix:
[0.1, 0.4, 1.1, 1.7]
[0, 0.2, 0.8, 1.4]
[0, 0, 0.4, 1.0]
[0, 0, 0, 0.3]
Root Matrix:
[0, 1, 2, 2]
[0, 1, 2, 2]
[0, 0, 2, 2]
[0, 0, 0, 3]
```

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

```
[ ] 
    if value < dp[i][j]:
        dp[i][j] = value
        root[i][j] = r

    return dp, root

keys = [10, 12, 16, 21]
freq = [4, 2, 6, 3]

cost, root = optimal_bst(keys, freq)

print("Minimum Cost:", cost[0][3])

print("Cost Matrix:")
for row in cost:
    print(row)

print("Root Matrix:")
for row in root:
    print(row)
```

```
✓ ... Minimum Cost: 26
    Cost Matrix:
    [4, 8, 20, 26]
    [0, 2, 10, 16]
    [0, 0, 6, 12]
    [0, 0, 0, 3]
    Root Matrix:
    [0, 0, 2, 2]
    [0, 1, 2, 2]
    [0, 0, 2, 2]
    [0, 0, 0, 3]
```

17. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: $\text{graph}[a]$ is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in $\text{graph}[1]$. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

[]



```
def cat_mouse_game(graph):
    if pc != 0:
        yield m, pc, 1
    else:
        # Parent was Mouse's turn
        for pm in graph[m]:
            yield pm, c, 0

while queue:
    m, c, turn, result = queue.popleft()

    for pm, pc, pt in parents(m, c, turn):
        if color[pm][pc][pt] != DRAW:
            continue

        if (pt == 0 and result == MOUSE) or \
            (pt == 1 and result == CAT):
            color[pm][pc][pt] = result
            queue.append((pm, pc, pt, result))
        else:
            degree[pm][pc][pt] -= 1

            if degree[pm][pc][pt] == 0:
                color[pm][pc][pt] = result
                queue.append((pm, pc, pt, result))

    return color[1][2][0]

graph1 = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]
print("Result:", cat_mouse_game(graph1))
```



... Result: 0

18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$. Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.



```
for (u, v), p in zip(edges, succProb):
    graph[u].append((v, p))
    graph[v].append((u, p))

probability = [0] * n
probability[start] = 1

pq = [(-1, start)]

while pq:
    neg_prob, u = heapq.heappop(pq)
    prob = -neg_prob

    if u == end:
        return prob

    for v, p in graph[u]:
        new_prob = prob * p

        if new_prob > probability[v]:
            probability[v] = new_prob
            heapq.heappush(pq, (-new_prob, v))

return 0

edges = [[0,1], [1,2], [0,2]]
prob = [0.5, 0.5, 0.2]

print(max_probability(3, edges, prob, 0, 2))
```

... 0.25

19. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

```
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

print("3 x 7:", unique_paths(3, 7))
print("3 x 2:", unique_paths(3, 2))
```

```
3 x 7: 28
3 x 2: 3
```

20. Given an array of integers `nums`, return the number of good pairs. A pair (i, j) is called good if `nums[i] == nums[j]` and $i \leq j$.

```
def good_pairs(nums):
    count = {}
    result = 0

    for num in nums:
        if num in count:
            result += count[num]

        count[num] = count.get(num, 0) + 1

    return result

print(good_pairs([1, 2, 3, 1, 1, 3]))
print(good_pairs([1, 1, 1, 1]))
```

```
4
6
```

21. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer distance `Threshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most distance `Threshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

```

        dist[i][j] = min(
            dist[i][j],
            dist[i][k] + dist[k][j]
        )

    answer = -1
    minimum = n

    for i in range(n):
        count = 0

        for j in range(n):
            if i != j and dist[i][j] <= threshold:
                count += 1

        if count <= minimum:
            minimum = count
            answer = i

    return answer

edges = [
    [0,1,3],
    [1,2,1],
    [1,3,4],
    [2,3,1]
]

print(findTheCity(4, edges, 4))

```

... 3

22. You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

]



```

        new_time = time + w

        if new_time < dist[v]:
            dist[v] = new_time
            heapq.heappush(pq, (new_time, v))

    answer = max(dist[1:])

    return -1 if answer == float('inf') else answer

print(network_delay_time(
    [[2,1,1], [2,3,1], [3,4,1]],
    4,
    2
))

print(network_delay_time(
    [[1,2,1]],
    2,
    1
))

print(network_delay_time(
    [[1,2,1]],
    2,
    2
))

```

... 2
1
-1